



A Low-power Programmable Machine Learning Hardware Accelerator Design for Intelligent Edge Devices

MINKWAN KEE and GI-HO PARK, Sejong University, Republic of Korea

With the advent of the machine learning and IoT, many low-power edge devices, such as wearable devices with various sensors, are used for machine learning-based intelligent applications, such as healthcare or motion recognition. While these applications are becoming more complex to provide high-quality services, the performance of conventional low-power edge devices with extremely limited hardware resources is insufficient to support the emerging intelligent applications. We designed a hardware accelerator, called an Intelligence Boost Engine (IBE), for low-power smart edge devices to enable the real-time processing of emerging intelligent applications with energy efficiency and limited programmability. The measurement results confirm that the proposed IBE can reduce the power consumption of the edge node device by 75% and achieve performance improvement in processing the kernel operations of applications such as motion recognition by 69.9 times.

CCS Concepts: • **Computer systems organization** → **Embedded systems**; *Parallel architectures*; • **Computing methodologies** → Machine learning;

Additional Key Words and Phrases: Machine learning, hardware accelerator, internet of things, edge computing, sensor fusion

ACM Reference format:

Minkwan Kee and Gi-Ho Park. 2022. A Low-power Programmable Machine Learning Hardware Accelerator Design for Intelligent Edge Devices. *ACM Trans. Des. Autom. Electron. Syst.* 27, 5, Article 51 (June 2022), 13 pages.

<https://doi.org/10.1145/3531479>

1 INTRODUCTION

Recently, **machine learning**– (ML) based services, such as image and the speech recognition, are used in many domains from autonomous driving to small devices such as artificial intelligence (AI) speaker. As these ML-based services began to help our lives, almost all small devices gathered data such as images and voices to provide services. The collected data are usually transferred to a server that has high computing power, and data are processed in the server for the ML in the conventional systems. However, this is becoming a heavier burden to the network resource and server, as the data collected by many edge devices have increased significantly. Even though transmitting

This work was supported by the Technology Innovation Program, 10049503, “Development of a low-power sensor processing MCU that consumes under 146uA/MHz for mobile smart device”, funded by the Ministry of Trade, industry & Energy (MI, Korea), and NRF-2018R1A2B6002534, NRF-2021R1H1A201341911, funded by National Research Foundation of Korea. Authors’ address: M. Kee and G.-H. Park (corresponding author), Sejong University, 209, Neungdong-ro, Gwangjin-gu, Seoul, Republic of Korea, 05006; emails: mkkee@sju.ac.kr, ghpark@sejong.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2022 Association for Computing Machinery.

1084-4309/2022/06-ART51 \$15.00

<https://doi.org/10.1145/3531479>

sensing data on the local gateway or smartphone enables the efficient processing of a large amount of data at a short distance, there are many issues such as unnecessary network traffic, increased energy consumption, and privacy protection have arisen [17]. As a solution of these issues, the local processing within edge devices that are closely integrated with sensing devices is emerged to provide services with fast and accurate response [5, 26]. However, local processing with the edge device also encounters various challenges such as achieving both the high performance and low power consumption. Various sensors have been installed around us to collect data on the surrounding environment in the internet of things (IoT) era, and the amount of sensing data from sensors is increasing to provide accurate information. In addition, many ML-based real-time services [7, 14, 24] that use sensors have also been developed, such as health care services that check the number of steps and emergency detection for patients or for workers in high-risk workspaces. To handle these data within edge devices, high performance with low-power consumption is becoming one of most important factors in local processing of edge devices that provide ML-based services.

A conventional sensing system uses low-power system on chip (SoC) such as sensor hubs to manage the sensing data, and such devices are placed near sensors or integrated with sensors. The sensor hub usually has ultra-low power cores, such as the ARM Cortex-M series core, to maintain low power consumption. The need for low power consumption was the most important factor of the design in edge devices, such as wearable devices, due to their battery-powered nature, but nowadays the computing power is also becoming an important factor to process advanced ML-based services. To improve performance, hardware accelerators have become very popular approaches, and graphic processing units, field programmable gate arrays, and **application-specific integrated circuits (ASICs)** are widely used in server systems for ML services. NVIDIA adds tensor cores for ML processing [21], and Google developed and utilized hardware accelerator called tensor processing unit (TPU) [15] for ML processing in its server systems. In addition, hardware for edge computing such as Google edge-TPU [3] and Intel Movidius [8] are also developed and utilized in edge devices. However, our target devices are more cost-sensitive and low-power devices that can be integrated with various sensors than devices that use edge-TPU or Movidius. Since hardware accelerators for existing edge devices were developed for Cortex-A-based devices, there is a large performance and cost difference from the cortex-M-based devices targeted in this work. We investigate design issues for hardware accelerators in intelligent edge devices and propose a hardware accelerator that has been designed to address these design issues.

2 DESIGN ISSUES FOR AN INTELLIGENT LOW-POWER EDGE DEVICE WITH VARIOUS SENSORS

To process intelligent IoT services, much higher performance is required even in embedded devices. However, embedded devices must operate from several days to several months without recharging or supplying power and should be very cheap, so it has very limited hardware resources to ensure low power consumption and cost effectiveness. Figure 1 shows the performance and power consumption characteristics of mobile application processors (AP), desktop central processing units (CPU), and low-power embedded cores. In general, mobile APs and wearable devices are included in edge devices, but even in edge devices, the gap in performance and power consumption is very large depending on target environment of applications. As shown in Figure 1, the difference in performance and energy consumption between Cortex-A core-based edge devices and low-power-oriented Cortex-M-based edge devices, which are usually considered as edge devices, is very large. For example, the Jetson Xavier NX of nVidia, which is a high-performance edge device, and the low-power-oriented Cortex-M-based **microcontroller unit (MCU)** targeted in this work, have a 100-fold difference in power consumption. This is because the conventional low-power-oriented MCU has performed very simple control operations, so it has been developed with the aim of

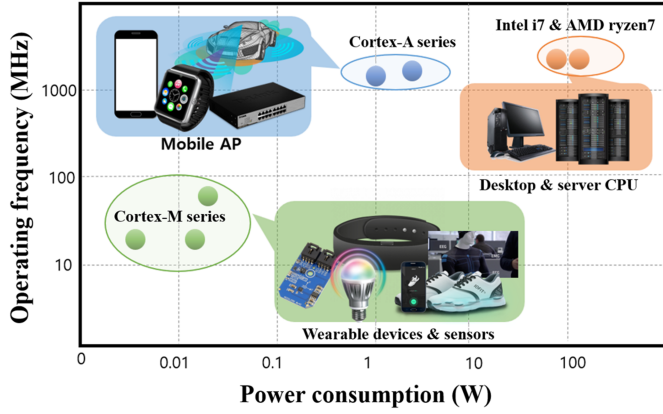


Fig. 1. Power consumption and operating frequency according to core types.

simple architecture for low power consumption and reliability. However, with the development of IoT technology, the requirement of higher performance for data processing has been increased even in low-power-oriented MCUs, so nowadays, the embedded MCU also faces challenging situations in terms of achieving high performance with low power consumption.

2.1 Requirement of High Performance and Ultra-low Power Consumption in Intelligent Edge Devices

In the past, edge devices with sensors have simply provided a service to collect data and send it to a local server or smartphone [6] to process the sensing data. However, emerging intelligent applications, such as motion recognition, require the smart edge device to locally perform machine learning algorithms, such as **support vector machine (SVM)**, **k-nearest neighbors (KNN)**, and sensor fusion algorithms with Kalman filters. The kernel operations of these applications are floating point **multiply-accumulate (MAC)** operations, and the performance requirement of these operations increases significantly as the accuracy and the complexity of the problem to be solved increase. Therefore, it is difficult to collect and process sensing data locally in a conventional ultra-low power core that generally has limited performance with several MHz to less than 200 MHz of operating frequency. In addition to performance requirements, battery-powered smart edge devices are much more power conscious than high-performance mobile and desktop environments. This is because devices must be operated for several days to several months with battery power. For this purpose, the most important consideration in battery-powered edge devices is to lower the power consumption, so battery-powered edge devices in the future should be developed to improve performance while maintaining low power consumption. To solve this performance gap between the performance of conventional low-power embedded cores and the performance requirements of the emerging intelligent applications, an innovative architecture for intelligent edge devices that can achieve both high performance and low power consumption is essential to support future intelligent IoT services. To achieve this goal, low-power edge devices achieve both high performance and low power consumption by utilizing dedicated hardware that has customized architecture for target applications [16, 18].

2.2 Hardware Accelerator with Limited Programmability to Support Both Power/Performance Efficiency and Programmability

Recently, various intelligent services, such as motion recognition and sensor fusion, have been adopting various intelligent edge devices. Usually, motion recognition services use machine

learning algorithms, such as SVM and KNN, for predicting the motion of users, and the sensor fusion technology is based on the Kalman filter to integrate the sensing data obtained from various sensors to improve the prediction accuracy of the orientation or rotation of a device. As the types and number of sensors integrated in intelligent edge devices has increased, the applications using these various sensors has become much more diverse. In addition to the diversity of the emerging intelligent applications, the machine learning algorithms that are used in intelligent edge devices are also evolving very rapidly. Therefore, some programmability should be provided to deal with rapid changes in intelligent services based on the various sensors and the evolving nature of machine learning algorithms.

It is a very natural and conventional approach to use a dedicated hardware accelerator, designed for the target application of a low-power edge device, to achieve both high performance and low power consumption. However, recent intelligent edge devices also require programmability to support various intelligent services and to support the evolving machine learning algorithms. Even though a dedicated hardware accelerator, solely designed for a specific target application, has excellent power-performance efficiency, it suffers from the lack of programmability. Considering these contrasting requirements, we designed a hardware accelerator with limited programmability that could achieve both power-performance efficiency and flexibility. The proposed hardware accelerator consists of a computing engine with an array of **processing elements (PE)** and a simple control unit to support the various common kernel operations of the target applications and machine learning algorithms.

3 RELATED WORKS

With the emergence of ML, many studies about the hardware accelerators and edge computing has begun. Research on hardware accelerators has been carried out in various fields and various environments, from mobile smartphones to high-performance servers. However, in the system where small chip size and low power consumption are important, such as in the wearable devices based on the **inertial measurement unit (IMU)** sensor, ASIC-based research for the specific application are mainly performed [16, 18, 20]. These works designed dedicated hardware accelerators that execute entire process of a specific machine learning algorithm.

Although this method has benefits in terms of high performance and power consumption, there is an important shortcoming that it is difficult to have the flexibility to perform other applications or updated target applications.

To overcome this shortcoming, we analyzed operations that are commonly shared by target applications and designed a hardware accelerator that processes major kernel operations based on the analysis. By accelerating common kernel operations that can be shared by other applications, we can provide some amount of flexibility with the proposed hardware accelerator. The proposed **Intelligence Boost Engine (IBE)** has 12 MACs and less than 1 KB of memory.

In recent studies such as in References [12, 23], hardware accelerators that meet both flexibility and performance were designed. However, the hardware accelerators designed in those papers targeted for relatively high-performance edge devices. For Eyeriss v2 of Reference [23], it is very efficient and delivers superior performance with 246 KB of memory and 384 MACs. Although this article targeted very low-power and cost effective edge device having very limited on chip resource, the IBE is designed to support a couple of essential applications of the power and resource limited edge device with only 12 MACs and less than 1 KB internal memory.

Most previous research that dealt with edge computing basically handled the computation process at the first gateway or the local server [10, 11] and aimed at reducing the burden of data transmission in wearable devices and sensor nodes [9, 22, 25]. However, in previous research, it is difficult to reduce the burden of continuously transmitting data in the wearable device and the

Target application	ratio	operational type	input data size
Sensor fusion 6dof	66%	matrix multiplication, matrix transpose	3x9, 9x3, 9x9
Sensor fusion 9dof	79%	matrix multiplication, matrix transpose	6x12, 12x6, 12x12
LIBSVM linear	73%	vector * vector & accumulation	1 ~ inf.
LIBSVM polynomial	78%	vector * vector & accumulation, vector +/- scalar, power	1 ~ inf.

(a) Analysis for kernel operation

Sensor fusion 6dof	Ratio of zero value in operands				
	Total Multi-plications	Zeros only in operand A	Zeros only in operand B	Zeros in operand A or B	
	Kalman gain 1	243	198	108	216
	Kalman gain 2	54	24	24	33
	Kalman gain 3	81	36	0	36
	Error covariance 1	243	108	198	216
	Error covariance 2	135	0	66	66
	Total	756	366(48.4%)	396(52.3%)	567(75%)

(b) Analysis for zero value ratio in operands of kernel function

Fig. 2. Kernel analysis of target applications.

sensor node. The burden of the edge device with various sensors will also increase as more accurate sensor data are required or a higher sampling rate is required. Different from previous studies, we design a dedicated hardware accelerator to perform processing or preprocessing of the application within wearable device or sensor node itself.

4 APPLICATIONS IN INTELLIGENT EDGE DEVICES

This section explains the execution characteristics and kernel operations of the target applications that will be accelerated by the proposed hardware accelerator. The required operations and operands are analyzed to design the hardware accelerator for the target applications.

4.1 Representative Target Applications

We selected the sensor fusion and motion recognition as the major target applications. Both applications operate with an IMU that consists of an accelerometer and a gyroscope. These applications are generally used in wearable devices, such as healthcare devices and smart bands. Since such devices have to operate from a few days to a few months, achieving low power consumption is very important, and sensor fusion technology for accurate sensing of behavior and simple machine learning for recognizing the user behavior are mainly used in these devices. Sensor fusion is an application designed to obtain accurate direction and rotation information of the users. In general, the Kalman filter, which is the kernel algorithm for sensor fusion, is used to secure accurate information about the rotation and direction of the user, and this information is derived from multiple sensors, because there are usually some errors in the sensing data from both the accelerometer and the gyroscope. Sensor fusion is becoming increasingly complex, as the sensors being used are increasing from a 6-axis sensor fusion, which uses an accelerometer and gyroscope, to a 9-axis sensor fusion, which also utilizes a magnetometer. Motion recognition applications utilize machine learning algorithms, such as the SVM and KNN algorithms, to predict the motion of a user. In general, only inference is performed in smart edge devices for target applications, because the training requires high performance and a large amount of storage that are better suited for server systems.

4.2 Major Kernel Operations of Target Applications

A 6-axis sensor fusion and a 9-axis sensor fusion based on the open-source NXP Sensor Fusion [2], and motion recognition based on the libSVM [4] linear and polynomial kernel, are built and used to analyze the kernel operations of the target applications. The execution time of kernel functions, the types of kernel operations, and the characteristics of the operands are analyzed for these target applications. Figure 2(a) shows the execution time ratio of the kernel operations for the target applications. The execution time of kernel operations over the total execution time of the application

is about 66–78% in target applications. The detailed analysis of the kernel codes for sensor fusion and SVM are shown in Figure 2(a). The kernel operations of the target applications are mainly constructed with matrix-matrix multiplication, and the size of the input data differs according to each application. The SVM polynomial has additional operations, such as scalar product, scalar addition, and power operation, in kernel operations. The acceleration of matrix-matrix multiplication is a very important operation, because various machine learning algorithms, such as KNN and **deep neural network (DNN)**, also use this operation. The sparsity of the operands is analyzed to determine the necessity of supporting the sparsity of the matrix, and this is presented in Figure 2(b). As Figure 2(b) shows, the operands of sensor fusion are constructed with a sparse matrix. It shows 75% zero values, on average, are included in operand A or operand B in the process of deriving the Kalman gain and error covariance. This also means that 75% of multiplications can be skipped in the matrix multiplication operation for these kernel operations. In contrast, in the case of the motion recognition application based on the SVM, there is no sparsity, because the features that are used in training and prediction are composed of non-zero values, such as the average and standard deviation of the sensing data.

5 INTELLIGENCE BOOSTING ENGINE

We designed a hardware accelerator to accelerate sensor fusion and the SVM-based motion recognition algorithm with limited programmability based on the analysis of target applications. The proposed hardware accelerator, called the IBE, is designed to support the general-purpose operations that are commonly used in the kernel operations of the target applications with limited programmability to achieve power-performance efficiency. The IBE accelerates kernel operations of target applications, so non-kernel operations are processed by embedded core. Figure 3 shows the overall architecture of the proposed IBE. The IBE has a PE array that consists of 12 MAC units for the parallel processing of target algorithms. There are several internal memory components in IBE: two 12×1 size buffers (Memory A) to store two rows of matrix A with dual buffering, a 12×12 size buffer (Memory B) to store matrix B, and result registers in PE to store the partial sum of results. We designed internal memories that reuse matrix B and the partial sum of results, so matrix B is loaded only once, and the partial sum is processed internally without any storing operation to the external memory. This structure significantly reduces data movement between the main memory and the internal memories of the accelerator and avoids unnecessary power consumption and delay that occurred by the external memory access. Zero-skipping methods were also devised in the proposed IBE to skip the multiplications of sparse matrix to ensure low power consumption and efficient processing. The proposed IBE has a zero-check bit array for the proposed zero-skipping scheme.

5.1 Zero-skipping Method for the IBE

The zero-skipping method is widely used in many machine learning hardware accelerators. Various hardware accelerators [19, 23] for neural network processing use zero-skipping methods based on the compressed sparse matrix formats such as Compressed sparse row or Compressed sparse column. Because this compressed sparse matrix format represents the non-zero values with each addressing information, a hardware accelerator can perform the computation on the non-zero values with the addressing information. Those zero-skipping methods based on compressed sparse matrix formats are usually very effective to reduce the amount of the data transfer from the main memory to the hardware accelerator and unnecessary operations of the hardware accelerator when the sparsity of the matrix is very high. If the matrix has many non-zero values, though, then the efficiency of those zero-skipping methods can be significantly decreased, because the addressing information becomes large as the number of the non-zero elements of the matrix increases.

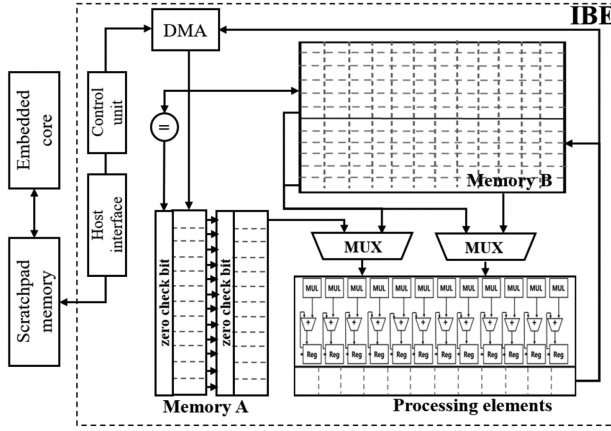


Fig. 3. Overview of proposed IBE.

As shown Figure 2(b), it was found that the sparsity of the target applications in this article was about 50% for each matrix. In this case, since addressing information must be needed for the 50% of non-zero values, achieving the benefits on cycles and energy consumption obtained by data compression and transmitting compressed data is very difficult. Besides, the accelerator that received the compressed matrix must decode the addressing information to process only non-zero values for the matrix multiplication, which requires additional operations and increased complexity of a control logic. Because the IBE is designed to perform machine learning acceleration with very limited hardware resources, we adopt the broadcasting-based matrix multiplication algorithm with only small size internal memory and 12 MAC units and a simple and efficient zero-skipping mechanism is also devised to consider the broadcasting-based matrix multiplication algorithm and the structure of the IBE.

We devised a zero-skipping mechanism that uses only one comparator to optimize the operation based on the sparsity of the matrix in the sensor fusion application. The simplest way to skip the zero value is to compare all the input data to PE and skip the MAC operation if any operand is zero. When the conventional matrix multiplication is used for performing the multiplication of two 12×12 matrices, it is necessary to check whether the value of 24 elements of the operands are zero or not in parallel to skip the multiplication for the zero element. As we investigated in Figure 2(b), this zero-skipping scheme can reduce multiplications by about 75%. Because the additional hardware cost and power consumption to check zero for all 24 values in parallel can be very high for low-power intelligent edge devices, we devised an efficient zero-skipping mechanism with broadcasting-based matrix multiplication methods that use only one comparator, as shown in Figure 4(a) [3]. This broadcasting-based multiplication method operates as shown in Figure 4(a). Each element in one row of matrix A is broadcasted, is multiplied with one row of matrix B, and then accumulates the partial sum in the result registers in PE. This routine is iterated until all elements of matrix A have been broadcasted and processed. After the process, one row of the result matrix is stored in the result registers in PE. We designed a zero-skipping mechanism that uses only one comparator to check the element of matrix A that is to be broadcast and skip the operation when the value of the element is zero. The hardware cost for the proposed zero-skipping mechanism is 24 times lower than a simple mechanism that checks the values of all operands and can skip 48.4% of multiplication operations. The operation of the proposed zero-skipping mechanism is shown in Figure 4(b). The comparator checks the values when a row of the matrix A is loaded into memory A and stores the value “1” into the zero-check bit register when the value of

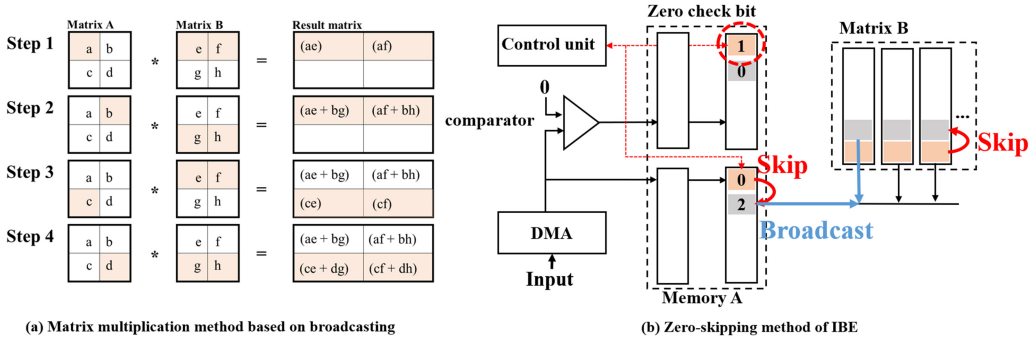


Fig. 4. Multiplication and zero-skipping methods of the IBE.

the matrix A element is zero. The control unit checks the zero-check bit register before broadcasting an element of memory A, and the broadcasting is skipped if the element is zero. The use of only one comparator in the IBE for zero skipping is designed to increase cost-effectiveness of the zero skipping method. Due to the characteristics of the broadcasting-based matrix multiplication method, while if an element of matrix A is 0, then the operation for the entire row of matrix B can be skipped; if an element of matrix B is 0, then only the MAC operation for the element having value 0 can be skipped. In addition, because the operation of the MAC also needs to be controlled individually based on the value of the element of matrix B for zero-skipping, we designed the zero skipping mechanism based on the value of the element in matrix A considering the tradeoff in terms of performance, energy consumption and cost. Based on this scheme, the proposed zero-skipping mechanism can deal with the sparsity of the matrix by checking only the elements of matrix A with a minimal hardware overhead and without any performance penalty.

5.2 Various Operation Modes for Programmability

The proposed IBE accelerates the various kernel operations in target algorithms with limited programmability. Therefore, we designed six operation modes that can be used in various algorithms. The operation modes consist of two types: general operation modes and algorithm-specific operation modes. The general operation modes can support other similar algorithms, while the algorithm-specific operation modes are designed to process the target algorithms more efficiently. The operation modes of the IBE that support the various operations are as follows:

- (1) Mode 0: $matrixA * matrixB$
- (2) Mode 1: $matrixA^T$
- (3) Mode 2: $matrixA * scalar$
- (4) Mode 3: $result + = vectorA * vectorB$
- (5) Mode 4: $(matrixA^T) * matrixB$
- (6) Mode 5: $result + = vectorA * vectorB$ (step 1)
 $result * scalar1 + scalar2$ (step 2)
 $result^{scalar3}$ (step 3)

Modes 0 to 3 are designed for general-purpose operations that can be used in other algorithms. Because operation modes 0 to 3 are general-purpose operations, they have relatively high flexibility so that they can support other algorithms as well as future updates of the target algorithms. Modes 4 and 5 are application-specific operation modes for target applications that combine general-purpose operation modes with some simple control signals. The operations for the mode 5 are performed as following sequences. In step 1 of mode 5, the result data of outputs from vector

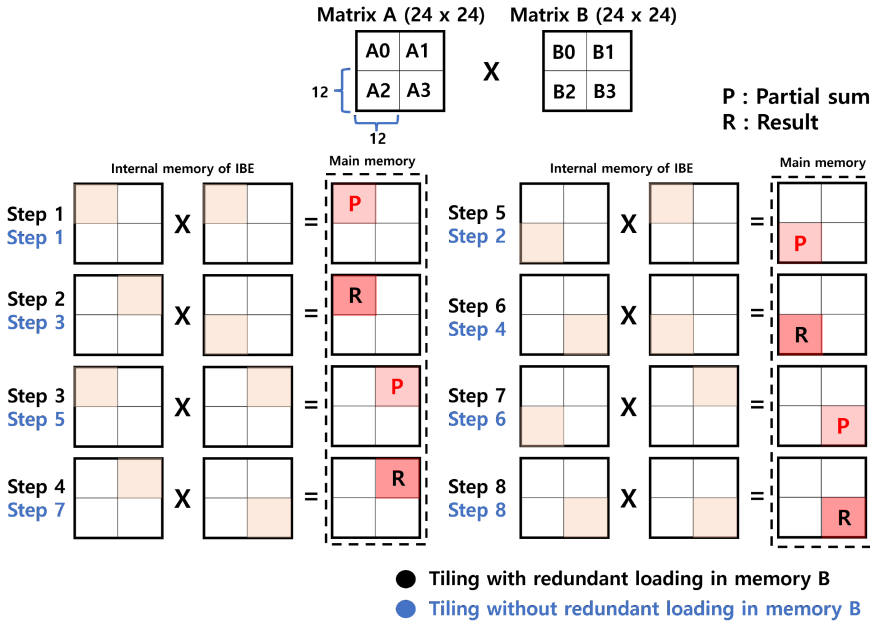


Fig. 5. Tiling process of matrix multiplication.

A * vector B are not returned and stored in memory B, while in mode 3, the result is stored into the result buffer at the bottom of the MAC unit of Figure 3. In addition, the values of scalar1 and scalar2 are loaded into Memory A, and the values of scalar3 are loaded into the control unit in step 1. The result of mode 3 and the scalar 1 are multiplied and stored in the memory B as in step 2. The result of the addition of scalar2 and the result values stored in memory B also stores back to memory B. In step 3, the power operation is performed through the multiplication operations repeatedly for the value of scalar3 and the result is returned to the scratchpad memory, and the control unit notifies to host embedded core that the operation has completed. These specified operation modes are meaningful, because it is possible to lower the overheads of data movement and communication overhead for control through the host interface compared to performing an operation using mode 0–3 functions several times.

The proposed IBE supports matrix multiplication in operation mode 0. Because the size of matrices to be processed can be different in various applications and the internal memory of IBE is limited, a tiling technique is commonly used to support various sizes of matrices. Figure 5 shows the process of performing matrix multiplication of 24×24 size using the tiling technique as an example. Since the IBE supports matrix multiplication of up to 12×12 , the matrix is divided into four sub-matrices and processed, and the processing steps are shown in Figure 5. First, matrix multiplication using a tiling technique, which is simple and easily understood is presented as black steps with the black-colored fonts. Because each step needs to multiply 12×12 matrix in a different area, matrix must be loaded into internal memory of the IBE for every steps. This causes a problem that the same data should be repeatedly loaded, loaded twice in this example, into the internal memory of the IBE. Fortunately, it is possible to remove some of the unnecessary data load just by changing the order of the operation steps as shown in blue steps with the blue-colored fonts in Figure 5. Redundant loading of the sub-matrix of the matrix B, second loading of the sub-matrix of the matrix B in this example, is removed easily in this case. The IBE processes the matrix multiplication of the sub-matrices in matrix A and B, then the partial sum is returned to the main memory.

Table 1. Evaluation Results for the IBE

Target algorithm	Kernel execution cycles (total cycles) * K cycles		Kernel speedup (total speedup)
	baseline	with IBE	
Sensor fusion 6dof	263 (409)	75 (221)	3.5 (1.9)
Sensor fusion 9dof	1,053 (1,339)	245 (531)	4.3 (2.5)
LIBSVM linear	4,012 (5,505)	163 (1661)	69.9 (3.5)
LIBSVM polynomial	5,228 (6,713)	165 (1665)	31.8 (4.0)
Target algorithm	Energy consumption (mJ)		Energy reduction
	baseline	with IBE	
Sensor fusion 6dof	0.12	0.07	45.9%
Sensor fusion 9dof	0.40	0.15	61.3%
LIBSVM linear	1.63	0.46	71.6%
LIBSVM polynomial	1.99	0.49	75.2%
Target algorithm	Network usage (bytes per transaction)		
	Server	Edge node with IBE	Data usage reduction
libSVM linear	1200	4	30,000%
libSVM polynomial	1200	4	30,000%

power down mode due to their simple architecture. Unlike centralized computing where data are constantly flooded, the edge device process the data with on-demand manner, then the edge device has done its work, it quickly switches to power down mode. For this reason, the fast data processing of the proposed IBE makes the duration that cores are in power down mode much longer, which plays a major role in increasing energy efficiency.

6.2 Network Traffic Reduction

Application processing at the edge node locally can play a significant role in reducing the power consumption of network. The amount of data to be transmitted can be significantly reduced, because the required data to a server is transmitted in a pre-processed form with the IBE instead of transmitting the raw data. Table 1 presents the amount of transmitted data in the motion-recognition application for both system configurations. The size of the data that should be transmitted is significantly reduced, and the communication frequency between the smart edge device and the server is also significantly reduced when the edge device has a limited size buffer. For example, if there is a 100 KB of buffer memory to store raw sensing data that are obtained by 9-axis sensors with 100 Hz sensing period, then it can only store the data for 28 seconds, while motion data that are locally processed in the intelligent edge device can be buffered for a maximum of 8,400 seconds. So, the communication frequency can be reduced about 300 times by the local processing with the proposed IBE. Because the communication frequency can vary according to the characteristic of applications, the reduction ratio can differ between applications. For example, in the case of a healthcare device that executes fall detection for a patient does not necessary to transmit the processed data to a server unless the result of full detection based on the sensing data is interpreted as a patient falls, the reduction in communication frequency can be reduced much more significantly in this application through the local processing with the IBE.

7 CONCLUSION

We investigated the design issues of a hardware accelerator for intelligent edge devices and proposed a hardware accelerator, called the IBE, to accelerate IMU-based applications, such as SVM

and sensor fusion with limited programmability. We designed the IBE with the purpose that the hardware accelerator can be adopted in many intelligent edge devices, such as smart bands or healthcare devices. The architecture of the IBE is designed to support the major kernel operations that are common in these applications based on limited programmability; so that it can be adopted with software update to future upgrades of target applications as algorithms of machine learning are rapidly progressing and being constantly updated. With the emerging of deep learning-based algorithms, a library such as ARM's CMSIS-NN [1] has been developed to utilize it in MCUs such as Cortex-M. Various DNN algorithms are also consist of huge amount of matrix multiplications, so it is expected the proposed IBE can be used for DNN, and we consider this as a future work. The proposed IBE has efficient hardware architecture to perform a broadcasting-based matrix multiplication with the proposed effective zero-skipping mechanism. The measurement results confirm that the proposed IBE can reduce the power consumption of the edge node device by 75% and achieve performance improvement in processing the kernel operations of applications such as motion recognition by 69.9 times.

REFERENCES

- [1] 2021. ARM CMSIS-NN. Retrieved October 26, 2021 from <https://developer.arm.com/documentation/102591/latest>.
- [2] 2021. NXP Sensor Fusion. Retrieved May 27, 2021 from <https://www.nxp.com/design/sensor-developer-resources/nxp-sensor-fusion:XTRSICSNSTLBOXX>.
- [3] S. Cass. 2019. Taking AI to the edge: Google's TPU now comes in a maker-friendly package. *IEEE Spectr.* 56, 5 (May 2019), 16–17. <https://doi.org/10.1109/MSPEC.2019.8701189>
- [4] C. C. Chang and C. J. Lin. 2011. LIBSVM - A library for support vector machines. *ACM Trans. Intell. Syst. Technol.* 27, 2, 3 (May 2011), 1–27. <https://doi.org/10.1145/1961189.1961199>
- [5] CISCO. 2015. *Fog Computing and the Internet of Things: Extend the Cloud to Where the Things are*. White Paper. Cisco.
- [6] A. Balasubramanian et al. 2013. *Efficiently Running Continuous Monitoring Applications on Mobile Devices Using Sensor Hubs*. Techreport. University of Washington, Intel.
- [7] A. Colao et al. 2013. Mime: Compact, low power 3D gesture sensing for interaction with head mounted displays. In *Proceedings of the 26th Annual ACM Symposium on User Interface Software and Technology (UIST'13)*. ACM, New York, NY, 227–236.
- [8] C. Marantos et al. 2018. Efficient support vector machines implementation on Intel/Movidius Myriad 2. In *Proceedings of the International Conference on Modern Circuits and Systems Technologies (MOCAS'T'18)*. IEEE.
- [9] D. Zordan et al. 2014. On the performance of lossy compression schemes for energy constrained sensor networking. *ACM Trans. Sens. Netw.* 11, 1 (August 2014), 1–34. <https://doi.org/10.1145/2629660>
- [10] I. Bisio et al. 2014. A smartphone-centric platform for remote health monitoring of heart failure. *Int. J. Commun. Syst.* 28, 11 (April 2014), 1753–1771. <https://doi.org/10.1002/dac.2778>
- [11] I. Bisio et al. 2016. Enabling IoT for In-Home rehabilitation accelerometer signals classification methods for activity and movement recognition. *IEEE IoT J.* 4, 1 (October 2016), 135–146. <https://doi.org/10.1109/JIOT.2016.2628938>
- [12] Kuo-Wei Chang et al. 2020. VWA: Hardware efficient vectorwise accelerator for convolutional neural network. *IEEE Trans. Circ. Syst.* 67, 1 (January 2020), 145–154. <https://doi.org/10.1109/TCSI.2019.2942529>
- [13] M. Hayashikoshi et al. 2014. Normally-off MCU architecture for low-power sensor node. In *Proceedings of the Asia and South Pacific Design Automation Conference (ASP-DAC'14)*. IEEE.
- [14] N. Abbas et al. 2017. Mobile edge computing: A survey. *IEEE IoT J.* 5, 1 (February 2017), 450–465. <https://doi.org/10.1109/JIOT.2017.2750180>
- [15] N. P. Jouppi et al. 2017. In-Datacenter performance analysis of a tensor processing unit. In *Proceedings of the International Symposium on Computer Architecture (ISCA'17)*. ACM.
- [16] Paolo Zicari et al. 2008. A matrix product accelerator for field programmable systems on chip. *Microprocess. Microsyst.* 32, 2 (2008), 53–67. <https://doi.org/10.1016/j.micpro.2007.05.002>
- [17] P. J. Soh et al. 2015. Wearable wireless health monitoring: Current developments, challenges, and future trends. *IEEE Microw. Mag.* 16, 4 (May 2015), 55–70. <https://doi.org/10.1109/MMM.2015.2394021>
- [18] S. Cruz et al. 2013. FPGA implementation of a sequential extended Kalman filter algorithm applied to mobile robotics localization problem. In *Proceedings of the IEEE 4th Latin American Symposium on Circuits and Systems (LASCAS'13)*. IEEE.
- [19] Song Han et al. 2016. EIE: Efficient inference engine on compressed deep neural network. In *Proceedings of the International Symposium on Computer Architecture (ISCA'16)*. ACM.

- [20] S. Huang et al. 2017. A flexible low-power machine learning accelerator for healthcare applications. In *Proceedings of the International Conference on Solid-State and Integrated Circuit Technology (ICSICT'17)*. IEEE.
- [21] S. Markidis et al. 2018. NVIDIA tensor core programmability, performance & precision. In *Proceedings of the International Parallel and Distributed Processing Symposium Workshops (IPDPSW'18)*. IEEE.
- [22] W. Hu et al. 2017. Toward joint compression-transmission optimization for green wearable devices: An energy-delay tradeoff. *IEEE IoT J.* 4, 4 (May 2017), 1006–1018. <https://doi.org/10.1109/JIOT.2017.2704605>
- [23] Yu-Hsin Chen et al. 2019. Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices. *IEEE J. Emerg. Select. Top. Circ. Syst.* 9, 2 (April 2019), 292–308. <https://doi.org/10.1109/JETCAS.2019.2910232>
- [24] Y. Zheng et al. 2014. Unobtrusive sensing and wearable devices for health informatics. *IEEE Trans. Biomed. Eng.* 61, 5 (May 2014), 1538–1554. <https://doi.org/10.1109/TBME.2014.2309951>
- [25] Y. Liang. 2011. Efficient temporal compression in wireless sensor networks. In *Proceedings of the Conference on Local Computer Networks*. IEEE.
- [26] W. Shi and S. Dustdar. 2016. The promise of edge computing. *Computer* 49, 5 (May 2016), 78–81. <https://doi.org/10.1109/MC.2016.145>

Received July 2021; revised April 2022; accepted April 2022